Проектирование БД «Фитнес»

Дубровин Т. Г. · Крехов Д. А. · Цибиров М. А.

Формат работы

Три учебные ветки A, B и C на одном условии. Для каждой показаны первая схема, замечания и исправленный вариант. Сравнение относится к устройству схем.

Исходное условие · вариант 8

Общий список услуг с ценами по видам услуг и квантам времени. Набор констант по максимальной загруженности каждого зала (в одном зале м.б. несколько услуг и одна услуга м.б. в неск. залах). Можно приобрести абонемент на некоторую услугу на определенный квант времени в определенный зал (количество абонементов ограничено вместимостью залов). Можно купить разовое посещение на определенную услугу на некоторый квант времени, если есть свободные места в залах. Безлимитных абонементов нет. Любое посещение привязано к услуге, залу и кванту времени.

• Поддержка оформления абонемента

• Поддержка оформления разового посещения (только на текущий день)

• Поддержка возможности отказаться от абонемента с возвратом части не потраченных денег

• Фин. отчет (за год);

• Запросы: Текущая загруженность залов; Прогноз на свободные места на определенную дату по квантам времени; Оценка возможности покупки абонемента.

Общие правила

Единый начальный запрос

Спроектируй реляционную схему базы данных по приведённому заданию. Верни один полный блок DBML для dbdiagram.io. Включи таблицы, поля, типы, ключи и связи. Обеспечь реализуемость операций и отчётов. Допущения укажи в комментариях DBML. SQL и код приложения не нужны. Далее передаётся полное исходное условие со страницы 1.

Абонемент

Фиксируем услугу, зал и квант, а также конечный список будущих дат. Каждое право относится к одному сеансу. Разовое посещение имеет одно право, его дата равна текущей дате клуба.

Расписание

Услуга допускается в зал через hall_service. Кванты внутри дня не пересекаются. Проверка совместимости услуги, зала, кванта тарифа и всех сеансов выполняется перед записью покупки.

Деньги

Цены версионируются по дате вступления в силу. Цена продажи фиксируется в purchase, её распределение по правам — в entitlement. Возврат покрывает будущие неиспользованные сеансы. Сумма возвратов ограничена суммой полученной оплаты.

Измерение загрузки

Фактическая загрузка — вошедшие и ещё не вышедшие посетители. Прогноз — вместимость минус действующие резервы. Для покупки абонемента проверяются все даты, даже если на первой свободно.

А · исходный DBML · 1

```
Table hall {
  id int [pk]
  capacity int [not null]
}

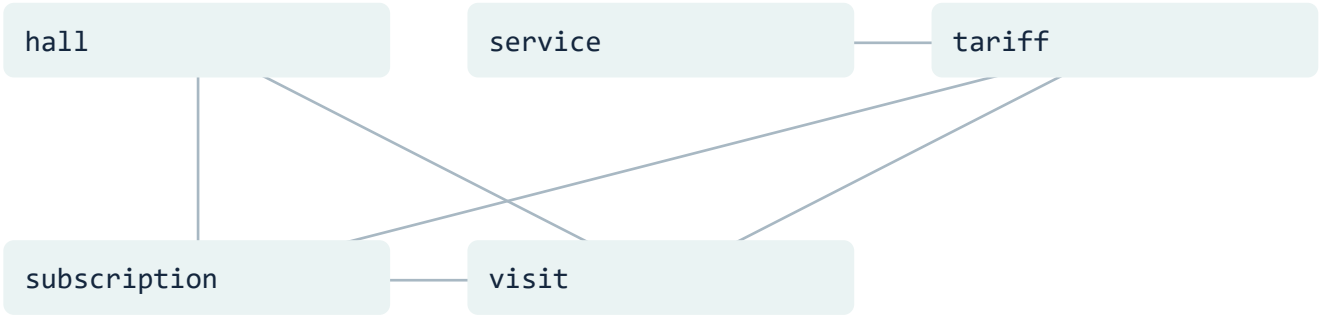
Table service {
  id int [pk]
  name varchar [not null]
}

Table tariff {
  id int [pk]
  service_id int [not null, ref: > service.id]
  quantum time [not null]
  price decimal [not null]
}

Table subscription {
  id int [pk]
  client_id int [not null]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  visits_left int [not null]
}

Table visit {
  id int [pk]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  subscription_id int [ref: > subscription.id]
  visit_day date [not null]
}
```

A · исходная схема




```
service.id  ←  tariff.service_id
hall.id  ←  subscription.hall_id
tariff.id  ←  subscription.tariff_id
hall.id  ←  visit.hall_id
tariff.id  ←  visit.tariff_id
subscription.id  ←  visit.subscription_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

A · Физические места

Проблема и последствия

У абонемента только остаток, поэтому будущие места не заняты. Посещение другой услуги в том же зале может исчерпать вместимость. Изменение цены в тарифе затрудняет расчёт суммы возврата.

Запрос на исправление

Создайте отдельные места зала и будущие посещения. Свяжите место с конкретным залом и интервалом. Зафиксируйте оплату, сумму каждого посещения и возврат. Сохрани полное исходное условие и верни полный DBML.

Исправление и контроль целостности

Действующий резерв уникален по (slot_id, seat_no), частичный индекс применяется при released_at IS NULL. Составные внешние ключи проверяют соответствие услуги, интервала и зала. Сортировать сеансы по ID и резервировать весь набор одной транзакцией.

После исправления

У продаж появляется конечный набор сеансов и неизменяемая цена. Финансовые события хранятся отдельно. Межтабличные правила и транзакции дополняют DBML.

А · исправленный DBML · 1

```
Table service {
  id bigint [pk]
  name varchar [not null]
}

Table hall {
  id bigint [pk]
  name varchar [not null]
}

Table hall_seat {
  hall_id bigint [not null, ref: > hall.id]
  seat_no int [not null]
  indexes { (hall_id, seat_no) [pk] }
}

Table quantum {
  id bigint [pk]
  starts_at time [not null]
  ends_at time [not null]
}

Table hall_service {
  hall_id bigint [not null, ref: > hall.id]
  service_id bigint [not null, ref: > service.id]
  indexes { (hall_id, service_id) [pk] }
}

Table tariff {
  id bigint [pk]
  service_id bigint [not null, ref: > service.id]
  quantum_id bigint [not null, ref: > quantum.id]
  product_kind varchar [not null]
  valid_from date [not null]
  amount_minor bigint [not null]
  indexes { (service_id, quantum_id, product_kind, valid_from) [unique] }
}
```

А · исправленный DBML · 2

```
Table hall_slot {
  id bigint [pk]
  hall_id bigint [not null, ref: > hall.id]
  quantum_id bigint [not null, ref: > quantum.id]
  local_day date [not null]
  indexes {
    (hall_id, quantum_id, local_day) [unique]
    (id, hall_id) [unique]
  }
}

Table offering {
  id bigint [pk]
  slot_id bigint [not null, ref: > hall_slot.id]
  service_id bigint [not null, ref: > service.id]
  indexes {
    (slot_id, service_id) [unique]
    (id, slot_id) [unique]
  }
}

Table client {
  id bigint [pk]
  name varchar [not null]
}

Table purchase {
  id bigint [pk]
  client_id bigint [not null, ref: > client.id]
  tariff_id bigint [not null, ref: > tariff.id]
  sold_at timestamp [not null]
  cancelled_at timestamp
  total_minor bigint [not null]
}
```

А · исправленный DBML · 3

```
Table entitlement {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  offering_id bigint [not null, ref: > offering.id]
  slot_id bigint [not null, ref: > hall_slot.id]
  hall_id bigint [not null, ref: > hall.id]
  seat_no int [not null]
  allocated_minor bigint [not null]
  released_at timestamp
  checked_in_at timestamp
  checked_out_at timestamp
  indexes { (purchase_id, offering_id) [unique] }
}

Table money_event {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  provider_key varchar [not null, unique]
  posted_at timestamp [not null]
  kind varchar [not null]
  amount_minor bigint [not null]
}
```

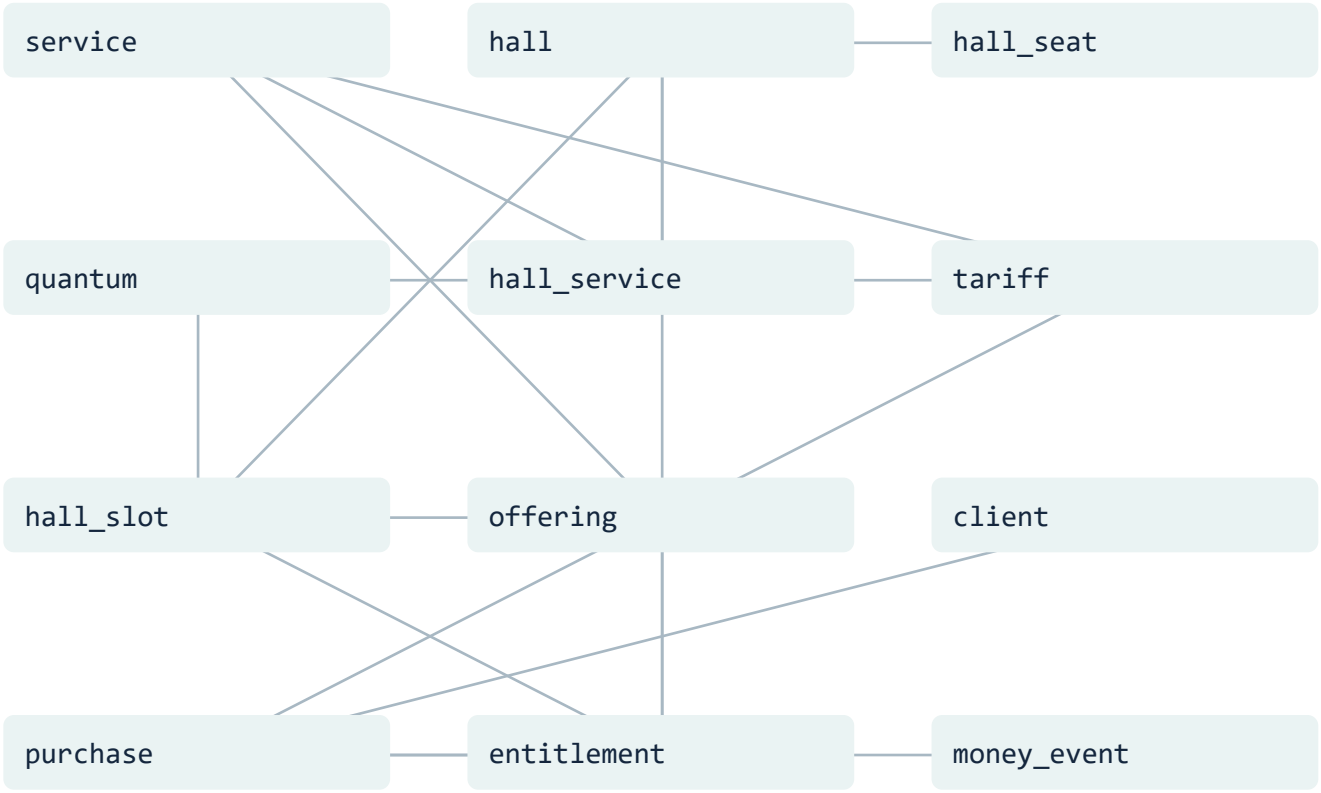
Составные связи · A

```
Ref: entitlement.(hall_id, seat_no) > hall_seat.(hall_id, seat_no)
Ref: entitlement.(slot_id, hall_id) > hall_slot.(id, hall_id)
Ref: entitlement.(offering_id, slot_id) > offering.(id, slot_id)
```

Назначение

Связи фиксируют три согласованных факта: услуга запланирована именно в указанном интервале, интервал принадлежит указанному залу, место входит в список мест этого зала.

А · исправленная схема




```
hall.id  ←  hall_seat.hall_id
hall.id  ←  hall_service.hall_id
service.id  ←  hall_service.service_id
service.id  ←  tariff.service_id
quantum.id  ←  tariff.quantum_id
hall.id  ←  hall_slot.hall_id
quantum.id  ←  hall_slot.quantum_id
hall_slot.id  ←  offering.slot_id
service.id  ←  offering.service_id
client.id  ←  purchase.client_id
tariff.id  ←  purchase.tariff_id
purchase.id  ←  entitlement.purchase_id
offering.id  ←  entitlement.offering_id
hall_slot.id  ←  entitlement.slot_id
hall.id  ←  entitlement.hall_id
purchase.id  ←  money_event.purchase_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

B · исходный DBML · 1

```
Table hall {
  id int [pk]
  capacity int [not null]
}

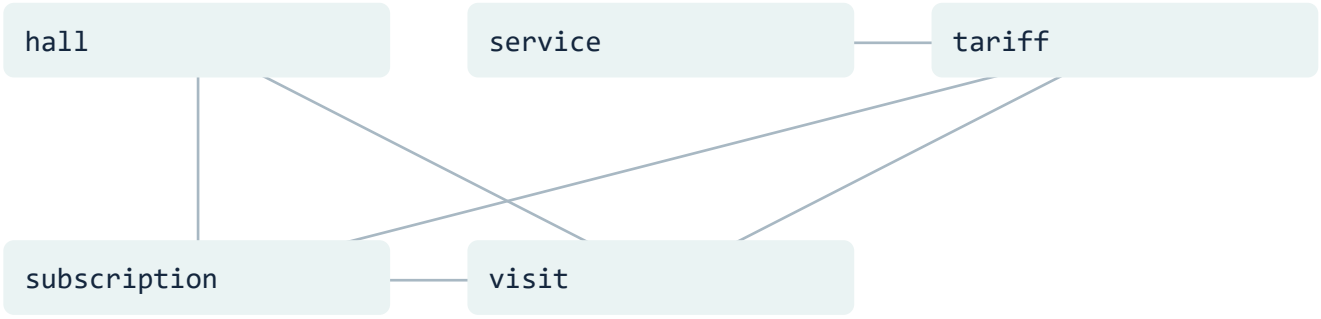
Table service {
  id int [pk]
  name varchar [not null]
}

Table tariff {
  id int [pk]
  service_id int [not null, ref: > service.id]
  quantum time [not null]
  price decimal [not null]
}

Table subscription {
  id int [pk]
  client_id int [not null]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  visits_left int [not null]
  paid_minor bigint [not null]
}

Table visit {
  id int [pk]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  subscription_id int [ref: > subscription.id]
  visit_day date [not null]
  service_capacity int [not null]
}
```

B · исходная схема




```
service.id  ←  tariff.service_id
hall.id  ←  subscription.hall_id
tariff.id  ←  subscription.tariff_id
hall.id  ←  visit.hall_id
tariff.id  ←  visit.tariff_id
subscription.id  ←  visit.subscription_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

B · Счётчик мест

Проблема и последствия

Ограничение вместимости на посещении и по отдельной услуге разъезжается между строками. Две покупки, прочитавшие одинаковый остаток, могут занять последнее место. paid_minor без проводок не отвечает на вопрос о возвратах прошлого года.

Запрос на исправление

Перенесите лимит на интервал зала и добавьте общий атомарный счётчик. Создавайте строки по каждому сеансу. Храните отдельные события оплаты и возврата. Сохрани полное исходное условие и верни полный DBML.

Исправление и контроль целостности

Увеличивать reserved_count условным UPDATE только при reserved_count < capacity_limit. Если UPDATE не затронул строку, откатывать весь заказ. При отмене сначала помечать право освобождённым один раз, затем уменьшать счётчик в той же транзакции. Периодически сверять счётчик с активными правами. seat_no не нужен, места взаимозаменяемы.

После исправления

У продаж появляется конечный набор сеансов и неизменяемая цена. Финансовые события хранятся отдельно. Межтабличные правила и транзакции дополняют DBML.

B · исправленный DBML · 1

```
Table service {
  id bigint [pk]
  name varchar [not null]
}

Table hall {
  id bigint [pk]
  capacity int [not null]
  name varchar [not null]
}

Table quantum {
  id bigint [pk]
  starts_at time [not null]
  ends_at time [not null]
}

Table hall_service {
  hall_id bigint [not null, ref: > hall.id]
  service_id bigint [not null, ref: > service.id]
  indexes { (hall_id, service_id) [pk] }
}

Table tariff {
  id bigint [pk]
  service_id bigint [not null, ref: > service.id]
  quantum_id bigint [not null, ref: > quantum.id]
  product_kind varchar [not null]
  valid_from date [not null]
  amount_minor bigint [not null]
  indexes { (service_id, quantum_id, product_kind, valid_from) [unique] }
}

Table hall_slot {
  id bigint [pk]
  capacity_limit int [not null]
  reserved_count int [not null]
  hall_id bigint [not null, ref: > hall.id]
  quantum_id bigint [not null, ref: > quantum.id]
  local_day date [not null]
  indexes {
    (hall_id, quantum_id, local_day) [unique]
    (id, hall_id) [unique]
  }
}
```

B · исправленный DBML · 2

```
Table offering {
  id bigint [pk]
  slot_id bigint [not null, ref: > hall_slot.id]
  service_id bigint [not null, ref: > service.id]
  indexes { (slot_id, service_id) [unique] }
}

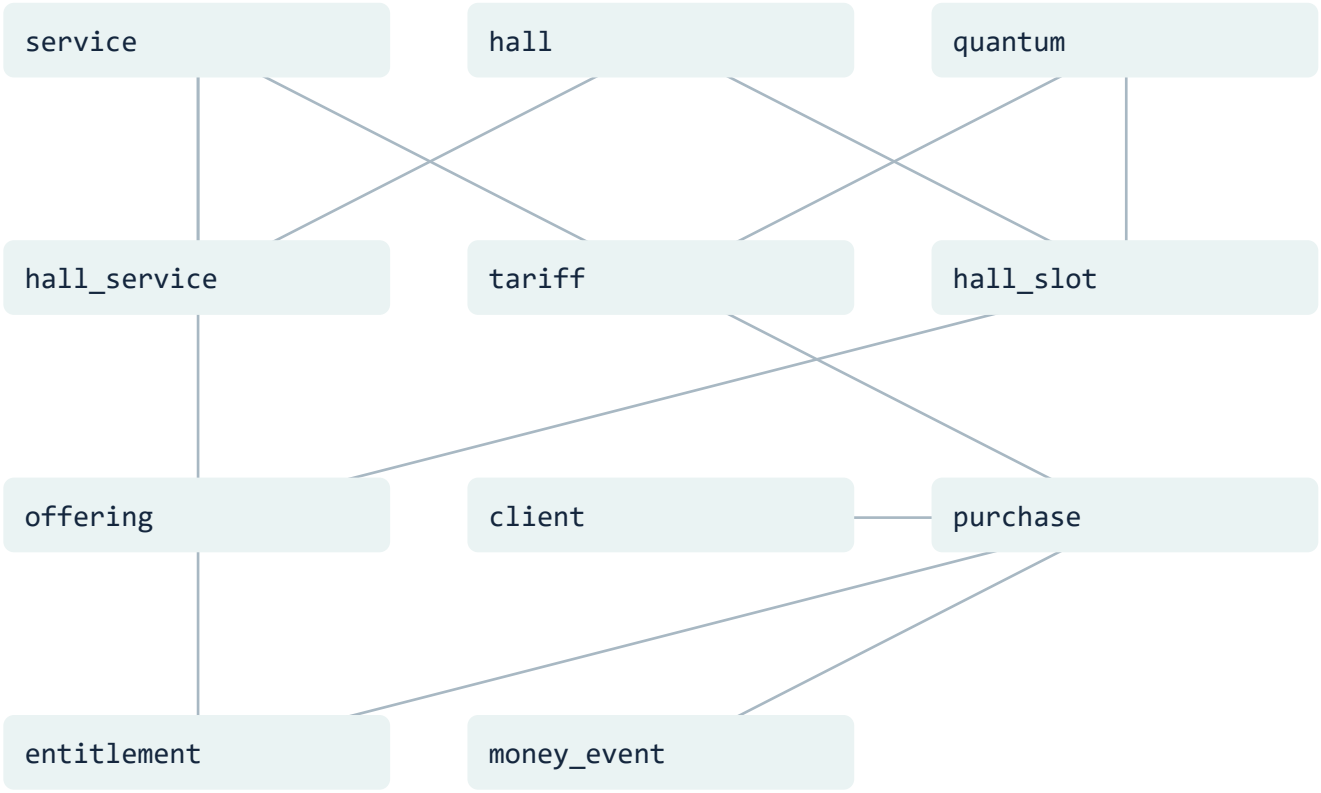
Table client {
  id bigint [pk]
  name varchar [not null]
}

Table purchase {
  id bigint [pk]
  client_id bigint [not null, ref: > client.id]
  tariff_id bigint [not null, ref: > tariff.id]
  sold_at timestamp [not null]
  cancelled_at timestamp
  total_minor bigint [not null]
}

Table entitlement {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  offering_id bigint [not null, ref: > offering.id]
  allocated_minor bigint [not null]
  released_at timestamp
  checked_in_at timestamp
  checked_out_at timestamp
  indexes { (purchase_id, offering_id) [unique] }
}

Table money_event {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  provider_key varchar [not null, unique]
  posted_at timestamp [not null]
  kind varchar [not null]
  amount_minor bigint [not null]
}
```

B · исправленная схема




```
hall.id  ←  hall_service.hall_id
service.id  ←  hall_service.service_id
service.id  ←  tariff.service_id
quantum.id  ←  tariff.quantum_id
hall.id  ←  hall_slot.hall_id
quantum.id  ←  hall_slot.quantum_id
hall_slot.id  ←  offering.slot_id
service.id  ←  offering.service_id
client.id  ←  purchase.client_id
tariff.id  ←  purchase.tariff_id
purchase.id  ←  entitlement.purchase_id
offering.id  ←  entitlement.offering_id
purchase.id  ←  money_event.purchase_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

C · исходный DBML · 1

```
Table hall {
  id int [pk]
  capacity int [not null]
}

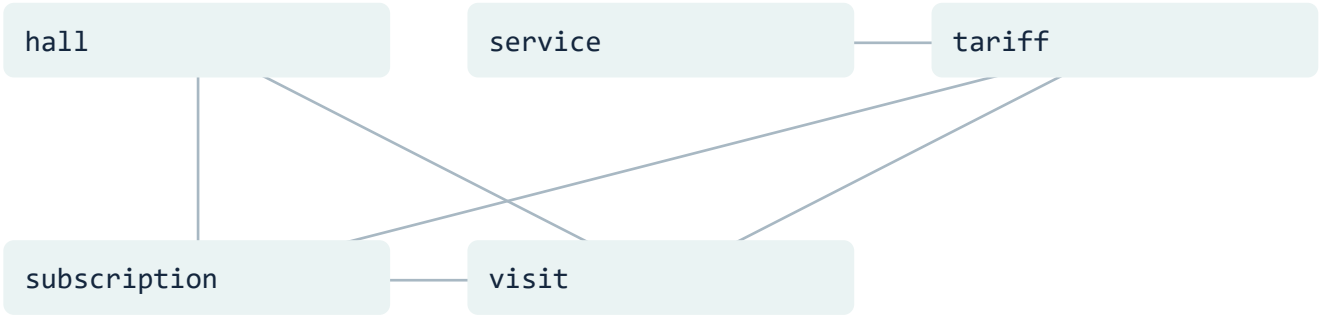
Table service {
  id int [pk]
  name varchar [not null]
}

Table tariff {
  id int [pk]
  service_id int [not null, ref: > service.id]
  quantum time [not null]
  price decimal [not null]
}

Table subscription {
  id int [pk]
  client_id int [not null]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  visit_limit int [not null]
  valid_until date [not null]
}

Table visit {
  id int [pk]
  hall_id int [not null, ref: > hall.id]
  tariff_id int [not null, ref: > tariff.id]
  subscription_id int [ref: > subscription.id]
  visit_day date [not null]
  cancelled boolean [not null]
}
```

C · исходная схема




```
service.id  ←  tariff.service_id
hall.id  ←  subscription.hall_id
tariff.id  ←  subscription.tariff_id
hall.id  ←  visit.hall_id
tariff.id  ←  visit.tariff_id
subscription.id  ←  visit.subscription_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

C · Журнал резервов

Проблема и последствия

Лимит и срок абонемента не определяют будущие сеансы. Булево поле cancelled не хранит время события. Если отказы и покупки фиксировать только текущим статусом, невозможно восстановить состояние на прошлую дату.

Запрос на исправление

Добавьте расписание и отдельное право на каждый сеанс. Сохраняйте события резервирования и освобождения, используйте общий лимит зала, отдельно храните расчёты с клиентом. Сохрани полное исходное условие и верни полный DBML.

Исправление и контроль целостности

Перед записью события RESERVED блокировать строку hall_slot. Считать права с RESERVED и без RELEASED. При заполнении слота откатывать все события покупки. RELEASED допускается только один раз и после RESERVED. Для прогноза на текущий момент учитывается актуальная проекция, для исторического среза фильтруются события по времени. seat_no не используется.

После исправления

У продаж появляется конечный набор сеансов и неизменяемая цена. Финансовые события хранятся отдельно. Межтабличные правила и транзакции дополняют DBML.

C · исправленный DBML · 1

```
Table service {
  id bigint [pk]
  name varchar [not null]
}

Table hall {
  id bigint [pk]
  capacity int [not null]
  name varchar [not null]
}

Table quantum {
  id bigint [pk]
  starts_at time [not null]
  ends_at time [not null]
}

Table hall_service {
  hall_id bigint [not null, ref: > hall.id]
  service_id bigint [not null, ref: > service.id]
  indexes { (hall_id, service_id) [pk] }
}

Table tariff {
  id bigint [pk]
  service_id bigint [not null, ref: > service.id]
  quantum_id bigint [not null, ref: > quantum.id]
  product_kind varchar [not null]
  valid_from date [not null]
  amount_minor bigint [not null]
  indexes { (service_id, quantum_id, product_kind, valid_from) [unique] }
}

Table hall_slot {
  id bigint [pk]
  capacity_limit int [not null]
  hall_id bigint [not null, ref: > hall.id]
  quantum_id bigint [not null, ref: > quantum.id]
  local_day date [not null]
  indexes {
    (hall_id, quantum_id, local_day) [unique]
    (id, hall_id) [unique]
  }
}
```

C · исправленный DBML · 2

```
Table offering {
  id bigint [pk]
  slot_id bigint [not null, ref: > hall_slot.id]
  service_id bigint [not null, ref: > service.id]
  indexes { (slot_id, service_id) [unique] }
}

Table client {
  id bigint [pk]
  name varchar [not null]
}

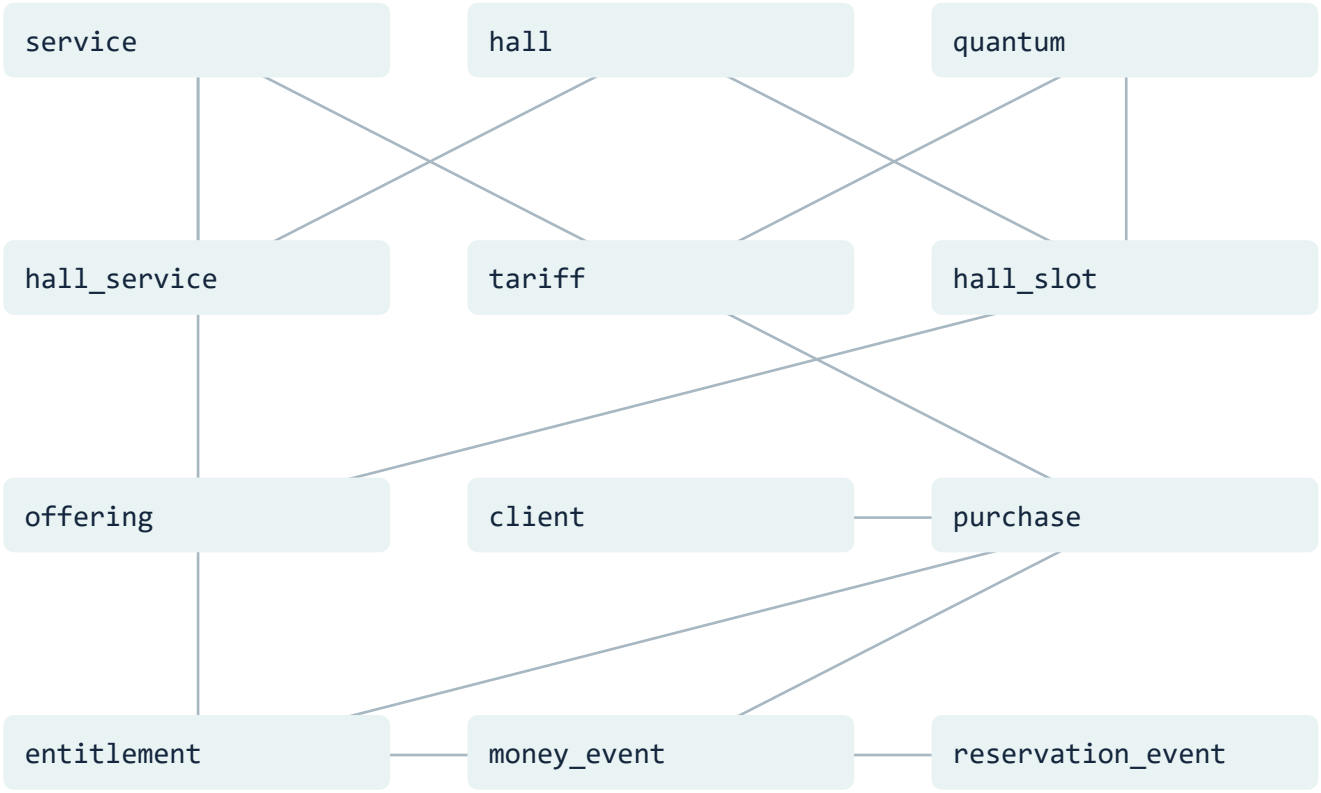
Table purchase {
  id bigint [pk]
  client_id bigint [not null, ref: > client.id]
  tariff_id bigint [not null, ref: > tariff.id]
  sold_at timestamp [not null]
  cancelled_at timestamp
  total_minor bigint [not null]
}

Table entitlement {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  offering_id bigint [not null, ref: > offering.id]
  allocated_minor bigint [not null]
  checked_in_at timestamp
  checked_out_at timestamp
  indexes { (purchase_id, offering_id) [unique] }
}

Table money_event {
  id bigint [pk]
  purchase_id bigint [not null, ref: > purchase.id]
  provider_key varchar [not null, unique]
  posted_at timestamp [not null]
  kind varchar [not null]
  amount_minor bigint [not null]
}

Table reservation_event {
  id bigint [pk]
  entitlement_id bigint [not null, ref: > entitlement.id]
  event_kind varchar [not null]
  occurred_at timestamp [not null]
  indexes { (entitlement_id, event_kind) [unique] }
}
```

C · исправленная схема




```
hall.id  ←  hall_service.hall_id
service.id  ←  hall_service.service_id
service.id  ←  tariff.service_id
quantum.id  ←  tariff.quantum_id
hall.id  ←  hall_slot.hall_id
quantum.id  ←  hall_slot.quantum_id
hall_slot.id  ←  offering.slot_id
service.id  ←  offering.service_id
client.id  ←  purchase.client_id
tariff.id  ←  purchase.tariff_id
purchase.id  ←  entitlement.purchase_id
offering.id  ←  entitlement.offering_id
purchase.id  ←  money_event.purchase_id
entitlement.id  ←  reservation_event.entitlement_id
```

Связи построены локально из внешних ключей DBML. Стрелка указывает на родителя.

Проверочные сценарии

A · Общая вместимость

В SQLite две услуги заняли два места одного зала в одном кванте. Повторный резерв занятого места и резерв несуществующего места отклонены. Место другого зала доступно. После освобождения второе место снова можно зарезервировать.

B · Абонемент на несколько дат

В прототипе первый сеанс имел место, второй был заполнен. Условное обновление второго сеанса не сработало. Откат вернул счётчик первого к исходному значению.

C · Освобождение через событие

После RESERVED свободных мест стало меньше. После RELEASED резерв исчез из расчёта. Повторная запись RELEASED отклонена составным ключом.

Годовой финансовый отчёт

Оплата 1200 и возврат 400 в 2026 году дают 800 чистого потока. Возврат 200 в январе 2027 года не изменил итог 2026 года. Расчёт выполнен в SQLite на примере проводок.

Границы проверки

Диаграммы построены локально из объявленных связей. Проверены изолированные правила на прототипах SQLite. Проверка параллельных покупок в целевой СУБД остаётся отдельным этапом.

Сравнение и выбор

A · Когда известны отдельные места

Частичный уникальный индекс защищает от двойного назначения конкретного места. Для оборудования, которое требуется идентифицировать, это полезно. Для обычного зала схема добавляет виртуальную нумерацию.

B · Когда места взаимозаменяемы

Проверка остатка проста, а обновление атомарно. Счётчик дублирует вычисляемый факт и требует сверки. Рекомендуемый вариант для условия с фиксированной вместимостью и без выбора места.

C · Когда важен аудит резервов

Порядок изменений можно восстановить по событиям. Стоимость решения — более сложная проекция и необходимость блокировки на уровне интервала. Полезен при требованиях к историческим срезам.

Итог

Все три исправленные схемы связывают посещение с услугой, залом и квантом, резервируют конечный набор дат и сохраняют финансовую историю. Отличается способ защиты вместимости. Для перехода к рабочей БД нужны ограничения на согласованность расписания, транзакции продажи и отмены и проверка параллельных покупок.